

**SCARLET**

System Container Application Release, Lifecycle & Environment Tool

MANUALE DI INSTALLAZIONE

Requisiti delle macchine per il deploy

Che cosa installare, dove e perché

SCARLET distribuisce applicazioni su server Oracle Linux remoti senza installare alcun agente. Questo manuale elenca, per ogni tipo di macchina coinvolta, il software da installare, l'utente da creare, le porte da aprire e i controlli da eseguire prima di dichiarare un server pronto. Include lo script di preparazione completo, la matrice dei flussi di rete e la checklist di consegna.

IN QUESTO DOCUMENTO

- Che cosa installare su ogni server di destinazione, con i comandi verificati
- Lo script di preparazione completo, dalla macchina vuota all'host pronto
- Utente dedicato, chiave SSH, approvazione del fingerprint, SELinux e firewall
- La matrice completa dei flussi di rete, con le porte obbligatorie e quelle opzionali
- Gli errori tipici con il messaggio esatto e il rimedio
- La checklist di consegna da allegare a ogni server

DESTINATARI Sistemisti, team infrastruttura, gestione firewall

VERSIONE SCARLET 1.0.0

DATA 14/09/2026

Indice

1. A chi serve questo documento

Le quattro famiglie di macchine

2. Server di destinazione: requisiti

Dimensionamento e sistema operativo

Pacchetti da installare

Comandi che SCARLET esegue davvero

3. Server di destinazione: installazione

Script completo, dalla macchina vuota

Utente dedicato e Podman rootless

Chiave SSH e authorized_keys

SELinux e firewall locale

4. Verifica prima di consegnare l'host

5. Varianti: Docker e Kubernetes

6. Il server SCARLET (control plane)

7. Postazioni e macchine di build

8. Matrice dei flussi di rete

9. Errori tipici e come risolverli

10. Checklist di consegna

Appendice A. Elenco completo dei binari richiesti

Appendice B. Documenti di riferimento

1. A chi serve questo documento

Questo manuale elenca, macchina per macchina, il software da installare prima che SCARLET possa distribuire applicazioni. È pensato per chi prepara i server: sistemisti, team infrastruttura, chi apre le regole sui firewall. Ogni comando è verificato su Oracle Linux 9.

Il punto di partenza

SCARLET è **senza agente**. Sui server gestiti non si installa alcun componente SCARLET: nessun demone, nessuna libreria Python, nessun pacchetto proprietario. Servono soltanto un accesso SSH con un utente dedicato, un runtime di container e una manciata di utilità di sistema già presenti in qualsiasi installazione minima.

Le quattro famiglie di macchine

Prima di scendere nel dettaglio conviene avere chiaro il perimetro. Le macchine coinvolte sono quattro e hanno requisiti molto diversi: solo la seconda riga richiede un intervento su ogni server, le altre si preparano una volta sola.

Macchina	Da installare	Quante	Capitolo
Server di destinazione gli Oracle Linux su cui girano le applicazioni	runtime di container + utilità di sistema + utente dedicato con chiave SSH	una per ogni server gestito	2, 3, 4
Server SCARLET il control plane	podman e podman-compose (o Docker Engine), certificato TLS, il file .env	una sola, centrale	6
PC degli operatori	nulla: un browser aggiornato	quante servono	7
PC o pipeline del team applicativo	il proprio strumento di build più lo script che confeziona il pacchetto	una per team	7

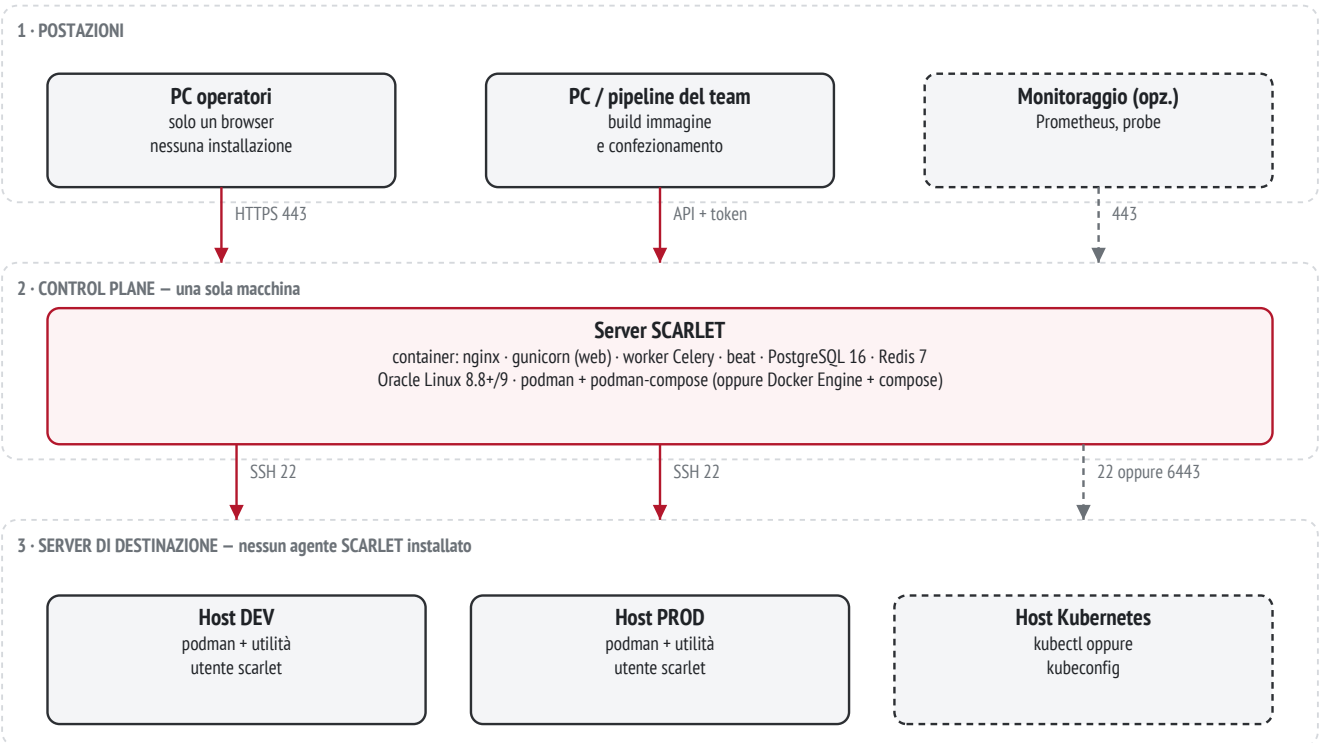


Figura 1 – Chi installa che cosa, e in quale direzione viaggia il traffico.

Perché conta la direzione

Le frecce hanno una sola direzione per un motivo preciso: i server di destinazione non devono mai poter raggiungere SCARLET. Non esistono callback, code in uscita o agenti che chiamano casa. Se un host viene compromesso, non ha alcun canale verso il control plane.

2. Server di destinazione: requisiti

Questo è il capitolo che riguarda **le macchine usate per il deploy**. Tutto ciò che segue vale per ciascun server, sia in DEV sia in PROD: la configurazione è identica, cambiano solo le regole di autorizzazione applicate lato SCARLET.

Dimensionamento e sistema operativo

Voce	Minimo	Consigliato	Note
Sistema operativo	Oracle Linux 8.8	Oracle Linux 9.x	vanno bene RHEL, Rocky e AlmaLinux della stessa famiglia
Architettura	x86_64	x86_64	l'immagine del container deve corrispondere all'architettura dell'host
CPU	1 vCPU oltre a quelle dell'applicazione	2 vCPU	SCARLET non consuma CPU sull'host se non durante il rilascio
RAM	512 MB liberi oltre all'applicazione	1 GB	il pre-flight confronta la memoria libera con quella dichiarata nel manifest
Disco su <code>/opt/scarlet</code>	5 GB	20 GB o più	vedi la formula qui sotto: pesa il numero di release conservate
SELinux	enforcing	enforcing	non va disattivato: i volumi sono etichettati con <code>:Z</code>
Orologio	sincronizzato (chrony)	sincronizzato	gli scarti di orario rendono illeggibili audit e log correlati

Lo spazio necessario si calcola così: **dimensione del pacchetto × release da conservare** più lo spazio delle immagini nello storage del runtime. Il numero di release conservate è un'impostazione di SCARLET (**Sistema > Impostazioni**, valore predefinito 5, la corrente e la precedente sono sempre mantenute). Le immagini non vengono eliminate automaticamente: se lo spazio è limitato, pianificare un `podman image prune` periodico sull'host.

Pacchetti da installare

Un solo comando copre il caso normale, cioè un host con Podman. I pacchetti sono tutti nei repository standard di Oracle Linux: non serve alcun repository aggiuntivo.

Installazione dei pacchetti

```
# Oracle Linux 9 – runtime consigliato (Podman rootless)
sudo dnf install -y podman podman-plugins fuse-overlayfs slirp4netns \
    tar gzip coreutils curl nmap-ncat

# openssh-server è già presente e attivo in ogni installazione standard
sudo systemctl enable --now sshd
```

Pacchetto	Perché serve	Obbligatorio
<code>podman</code> (4.x, meglio 5.x)	esegue i container; SCARLET rileva da solo se è rootless o rootful	sì, salvo che si usi Docker o Kubernetes
<code>podman-plugins</code> , <code>fuse-overlayfs</code> , <code>slirp4netns</code>	rete e storage in modalità rootless	sì con Podman rootless
<code>tar</code> , <code>gzip</code>	estrazione del pacchetto di rilascio	sì
<code>coreutils</code>	fornisce <code>sha256sum</code> , <code>ln</code> , <code>mv</code> , <code>df</code> , <code>free</code> e gli altri comandi di base	sì, già presente
<code>curl</code>	health check HTTP eseguiti dall'host su <code>127.0.0.1</code>	sì, se l'applicazione ha un health check HTTP
<code>nmap-ncat</code> (fornisce <code>nc</code>)	health check TCP e verifica delle porte in pre-flight	sì, se l'applicazione ha un health check TCP
<code>openssh-server</code>	l'unico canale di accesso di SCARLET	sì, già presente

Che cosa NON va installato

Non serve installare Python, né un agente, né alcun pacchetto di SCARLET. Non serve **sudo**: l'utente dedicato lavora esclusivamente dentro la propria home e dentro **/opt/scarlet**.

Comandi che SCARLET esegue davvero

SCARLET non apre una shell interattiva e non costruisce righe di comando concatenando testo. Ogni comando è una lista di argomenti il cui eseguibile deve appartenere a una allow-list compilata nel codice. Questo è anche il modo più onesto di elencare i requisiti: se un binario non è in questa lista, SCARLET non può eseguirlo, e se manca sull'host il relativo passo fallisce con un errore esplicito. L'elenco completo è nell'Appendice A.

Esempio reale

```
# tracciato tipico di un rilascio, visibile nella pagina del deployment
mkdir -p /opt/scarlet/applications/customer-api/releases
sha256sum -- /opt/scarlet/applications/customer-api/staging/2.5.0.ab12cd34.scarlet.tar.gz
tar --no-same-owner --no-same-permissions --no-overwrite-dir -xzf ... -C ...
mv -T -- ../staging/2.5.0.extract ../releases/2.5.0
podman pull registry.example.internal/team/customer-api:2.5.0
ln -sf -- ../releases/2.5.0 ../current.tmp && mv -T -- ../current.tmp ../current
podman run -d --name customer-api --label scarlet.application=customer-api ...
curl -k -s -S -o /dev/null -w %{http_code} --max-time 5 http://127.0.0.1:8080/health
```

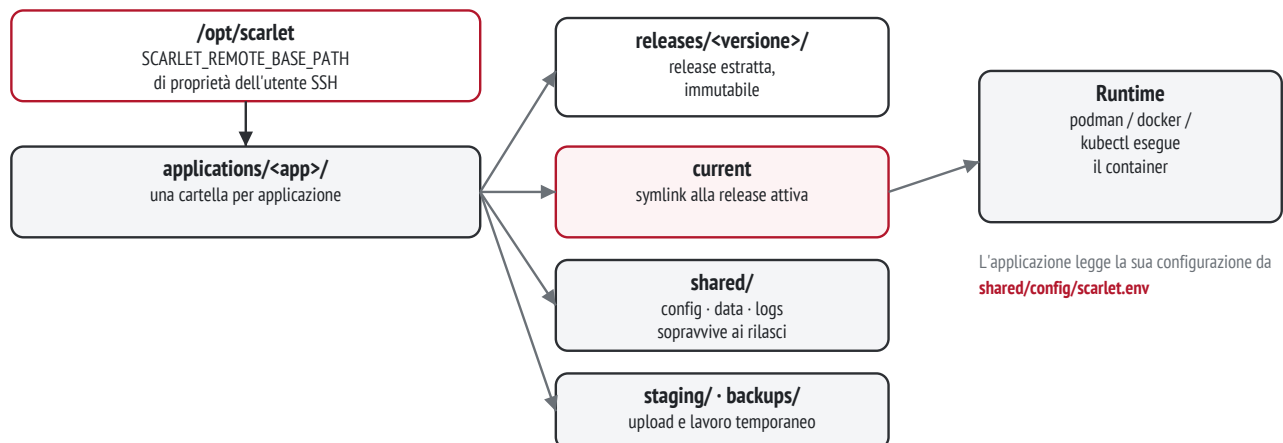


Figura 2 – Il layout che SCARLET crea sotto /opt/scarlet. Tutto il resto del filesystem non viene toccato.

3. Server di destinazione: installazione

La preparazione di un host richiede pochi minuti e si fa una volta sola. Qui trovi prima lo script completo da copiare, poi la spiegazione di ogni singolo passo.

Script completo, dalla macchina vuota

```
provisioning-host.sh
```

```
#!/bin/bash
# Preparazione di un server di destinazione SCARLET – Oracle Linux 9
# Eseguire come root (o con sudo) una sola volta per host.
set -euo pipefail

SCARLET_USER=scarlet
BASE_PATH=/opt/scarlet
SCARLET_SERVER_IP=10.10.0.5      # indirizzo del server SCARLET

# 1. pacchetti
dnf install -y podman podman-plugins fuse-overlayfs slirp4netns \
    tar gzip coreutils curl nmap-ncat

# 2. utente dedicato, senza sudo
id -u "$SCARLET_USER" >/dev/null 2>&1 || useradd -m -s /bin/bash "$SCARLET_USER"

# 3. namespace utente per Podman rootless (se non già assegnati)
grep -q "^${SCARLET_USER}:" /etc/subuid || \
    usermod --add-subuids 100000-165535 --add-subgids 100000-165535 "$SCARLET_USER"

# 4. i container sopravvivono alla chiusura della sessione SSH
loginctl enable-linger "$SCARLET_USER"

# 5. directory base
mkdir -p "$BASE_PATH"
chown "$SCARLET_USER:$SCARLET_USER" "$BASE_PATH"
chmod 750 "$BASE_PATH"

# 6. chiave pubblica di SCARLET (incollare qui la chiave generata sul server SCARLET)
install -d -m 700 -o "$SCARLET_USER" -g "$SCARLET_USER" "/home/$SCARLET_USER/.ssh"
cat > "/home/$SCARLET_USER/.ssh/authorized_keys" <<KEY
from="${SCARLET_SERVER_IP}",no-agent-forwarding,no-port-forwarding,no-X11-forwarding ssh-ed25519
AAAA... scarlet
KEY
chown "$SCARLET_USER:$SCARLET_USER" "/home/$SCARLET_USER/.ssh/authorized_keys"
chmod 600 "/home/$SCARLET_USER/.ssh/authorized_keys"
restorecon -Rv "/home/$SCARLET_USER/.ssh" || true

# 7. firewall: SSH solo dal server SCARLET, più le porte delle applicazioni
firewall-cmd --permanent --add-rich-rule=\
    "rule family=ipv4 source address=${SCARLET_SERVER_IP}/32 service name=ssh accept"
firewall-cmd --permanent --add-port=8080/tcp      # porta dell'applicazione
firewall-cmd --reload

# 8. fingerprint da confrontare in SCARLET al momento dell'approvazione
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Utente dedicato e Podman rootless

L'utente (chiamato **scarlet** per convenzione, il nome è configurabile per ogni host) è un utente Linux ordinario e senza privilegi. Tre dettagli fanno la differenza fra un host che funziona e uno che si comporta in modo imprevedibile.

- **subuid e subgid**: Podman rootless mappa gli utenti del container su un intervallo di UID dell'host. Se l'intervallo manca, il primo **podman run** fallisce con **newuidmap: Operation not permitted**. Su Oracle Linux 9 **useradd** di norma li assegna da solo: il controllo nello script è una rete di sicurezza.
- **linger**: senza **loginctl enable-linger** systemd chiude la sessione utente quando SCARLET chiude la connessione SSH, e con essa i container appena avviati. È l'errore più frequente e il più difficile da diagnosticare, perché il deployment risulta riuscito e l'applicazione sparisce qualche secondo dopo.
- **porte basse**: in modalità rootless le porte sotto la 1024 non sono utilizzabili. O si sceglie una porta applicativa uguale o superiore a 1024, oppure si abbassa il limite con **sysctl net.ipv4.ip_unprivileged_port_start=80**, rendendolo persistente in **/etc/sysctl.d/**.

Niente sudo, niente root

SCARLET non usa mai **sudo** e non lo richiede. Se la vostra policy impone Podman rootful, l'accesso SSH come root **non è supportato**: va usato un utente che possa raggiungere il socket rootful secondo le regole aziendali. La modalità consigliata, e quella verificata, è rootless.

Chiave SSH e authorized_keys

La chiave privata viene generata **sul server SCARLET** (ed25519) e caricata nella pagina **Sicurezza > Credenziali**, dove viene cifrata con Fernet prima di essere scritta nel database. Sull'host si installa solo la parte pubblica. L'autenticazione a password è supportata per il primo avvio ma sconsigliata: la rotazione di una chiave si fa aggiungendo una credenziale nuova, che disattiva la precedente e lascia traccia nell'audit.

Le restrizioni in **authorized_keys** non sono decorative. Il prefisso **from="10.10.0.5"** impedisce l'uso della chiave da qualsiasi altro indirizzo, e le tre opzioni **no-*** tolgono alla sessione la possibilità di aprire tunnel. Anche una chiave sottratta diventa così poco utile a un attaccante.

Approvazione del fingerprint dell'host

In produzione SCARLET rifiuta di collegarsi a un host la cui chiave non sia stata approvata da un amministratore (**SCARLET_SSH_HOST_KEY_POLICY=strict**). La procedura corretta prevede di leggere il fingerprint **sulla console del server**, non tramite la rete, e di confrontarlo con quello mostrato dalla console web.

Lettura fuori banda

```
# sulla console del server di destinazione
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
256 SHA256:5t0kV... root@app01 (ED25519)
```

Quando il fingerprint cambia

Cambiare hostname, indirizzo IP o porta SSH di un host invalida l'approvazione: la chiave va riapprovata. Una chiave diversa da quella approvata blocca l'host, lo segna OFFLINE e genera un evento di sicurezza CRITICAL. È il comportamento voluto: prima di riapprovare occorre accertarsi che il server sia stato davvero reinstallato.

SELinux e firewall locale

SELinux resta in modalità **enforcing**. SCARLET monta i volumi dei container con l'opzione **:Z**, che delega a Podman la rietichettatura, quindi non serve alcuna deroga. Se compare un diniego, la risposta non è disattivare SELinux ma correggere l'etichetta.

```
ausearch -m avc -ts recent          # che cosa è stato negato
restorecon -Rv /opt/scarlet         # rimette a posto le etichette
restorecon -Rv /home/scarlet/.ssh   # tipico dopo aver copiato authorized_keys a mano
```

Sul firewall dell'host servono due sole cose: la porta SSH raggiungibile **solo** dal server SCARLET e le porte con cui gli utenti raggiungono le applicazioni. Nient'altro va aperto in ingresso.

4. Verifica prima di consegnare l'host

Questi controlli si eseguono sull'host come utente `scarlet` e richiedono meno di un minuto. Se tutti danno l'esito atteso, la registrazione in SCARLET andrà a buon fine al primo tentativo.

Controlli sull'host

```
sudo -iu scarlet          # entrare nel contesto dell'utente dedicato

podman info --format '{{.Host.Security.Rootless}}'    # atteso: true
podman run --rm docker.io/library/busybox echo ok    # atteso: ok
for x in tar gzip sha256sum curl nc ln readlink df free nproc; do command -v $x >/dev/null ||
echo "MANCA $x"; done
test -w /opt/scarlet && echo 'scrittura ok'
loginctl show-user scarlet --property=Linger          # atteso: Linger=yes
grep '^scarlet:' /etc/subuid /etc/subgid              # atteso: due righe
getenforce                                            # atteso: Enforcing
```

Dal server SCARLET, invece, si verifica il canale end-to-end:

```
ssh -i /percorso/chiave -o BatchMode=yes scarlet@app01 'id; podman --version'
# deve rispondere senza chiedere password e senza chiedere conferma del fingerprint
```

L'ultima verifica si fa dalla console web e coincide con le azioni **Test connessione** e **Discover** nella pagina dell'host. La prima apre la connessione e misura la latenza, la seconda raccoglie sistema operativo, runtime, versione, memoria e disco, e compila la scheda dell'host. Se Discover riporta runtime **NONE**, il runtime non è installato oppure l'utente non riesce a eseguirlo.

Controllo	Esito atteso	Se fallisce
Test connessione	ONLINE, con latenza in millisecondi	chiave, utente, firewall o fingerprint non approvato
Discover	sistema operativo e runtime rilevati, disco e memoria valorizzati	runtime assente o non eseguibile dall'utente
Stato chiave host	APPROVED	eseguire Scan host key e approvare dopo il confronto fuori banda
Pre-flight di un deployment di prova	tutti i controlli verdi	leggere il controllo rosso: indica esattamente la causa

5. Varianti: Docker e Kubernetes

Podman è la scelta consigliata su Oracle Linux, ma gli altri due runtime sono supportati con lo stesso identico flusso di lavoro: cambia l'adattatore usato da SCARLET, non la procedura operativa.

Docker Engine

```
# repository ufficiale Docker su Oracle Linux
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin \
    tar gzip curl nmap-ncat
sudo systemctl enable --now docker
sudo usermod -aG docker scarlet # l'utente deve poter parlare con il socket
```

Il prezzo del gruppo docker

Appartenere al gruppo `docker` equivale, di fatto, ad avere root sull'host: chi può parlare con il socket può montare qualunque percorso dentro un container. È la ragione principale per cui su Oracle Linux consigliamo Podman rootless, che non richiede alcun privilegio aggiuntivo.

Kubernetes

Per i cluster ci sono due modalità. Nella prima l'host registrato in SCARLET è una macchina con `kubectl` già configurato, e SCARLET lo comanda via SSH esattamente come fa con Podman: serve solo la porta 22. Nella seconda SCARLET parla direttamente con l'API server usando un kubeconfig caricato come credenziale, e in quel caso occorre aprire la 6443 dal server SCARLET verso il cluster.

Modalità	Da installare sull'host	Porte	Quando sceglierla
kubectl via SSH	<code>kubectl</code> 1.27+ con kubeconfig valido per l'utente <code>scarlet</code>	22/tcp	quando esiste già una bastion host verso il cluster
API diretta	nessuna macchina intermedia: il kubeconfig è una credenziale in SCARLET	6443/tcp	quando il control plane raggiunge l'API server in rete

In modalità API non va installato **nulla**, da nessuna parte: non serve una macchina intermedia, non serve un utente Linux, non c'è un fingerprint da approvare e non viene creata alcuna directory sui nodi. SCARLET applica gli oggetti nel namespace e pubblica la configurazione come Secret. È la modalità consigliata quando il control plane raggiunge l'API server in rete.

Permessi richiesti alla credenziale

Un Role limitato al namespace, non `cluster-admin`. Serve `get list watch create update patch delete` sugli oggetti che il pacchetto installa (Deployment, Service, ConfigMap, Secret, Ingress e simili), `get list` sui pod, `get` su `pods/log` per i log e `get list` sugli eventi, che sono ciò che spiega un rollout fallito. Solo se SCARLET deve creare il namespace serve anche `get create` su `namespaces`, che è di ambito cluster.

Quello che un pacchetto non può installare

Gli oggetti RBAC (`ClusterRole`, `ClusterRoleBinding`) e le definizioni di risorse personalizzate non possono essere installati da un pacchetto: assegnarli è competenza del team di piattaforma, non di un rilascio applicativo. Un pacchetto che li contiene viene respinto con un messaggio che nomina il tipo.

6. Il server SCARLET (control plane)

Una sola macchina ospita l'intero control plane. Tutti i componenti girano come container di un unico compose: non si installa Python, né PostgreSQL, né nginx sul sistema operativo.

Voce	Minimo	Consigliato
CPU / RAM	2 vCPU / 4 GB	4 vCPU / 8 GB
Disco	40 GB	100 GB o più, con volume separato per <code>/var/lib/scarlet</code>
Sistema operativo	Oracle Linux 8.8, SELinux enforcing	Oracle Linux 9.x
Rete in ingresso	443/tcp dagli operatori	443/tcp; la 80 solo per il redirect
Rete in uscita	22/tcp verso ogni host gestito	più 587/tcp se si abilitano le e-mail

Installazione del control plane

```
# 1. runtime e utilità
sudo dnf install -y podman podman-compose git tar policycoreutils-python-utils firewalld

# 2. utente e directory dei dati
sudo useradd -r -m -d /opt/scarlet-server -s /bin/bash scarlet
sudo mkdir -p /var/lib/scarlet/{postgres,redis,data,nginx-logs} /opt/scarlet-server/tls
sudo chown -R scarlet:scarlet /opt/scarlet-server /var/lib/scarlet

# 3. configurazione: due chiavi DIVERSE, da custodire nel vault aziendale
cd /opt/scarlet-server && cp .env.example .env && chmod 600 .env
podman run --rm scarlet:1.0.0 flask scarlet gen-key # SCARLET_SECRET_KEY
podman run --rm scarlet:1.0.0 flask scarlet gen-key # SCARLET_CREDENTIAL_ENCRYPTION_KEY

# 4. avvio come servizio di sistema
sudo cp deployment/systemd/scarlet.service /etc/systemd/system/
sudo systemctl daemon-reload && sudo systemctl enable --now scarlet

# 5. verifica
curl -k https://scarlet.example.internal/api/ready
```

Le due chiavi non si recuperano

Senza `SCARLET_CREDENTIAL_ENCRYPTION_KEY` un backup del database è inutilizzabile: le credenziali SSH e i segreti non sono più decifrabili. Le due chiavi vanno nel vault aziendale **prima** di mettere il sistema in esercizio, e `SCARLET_INITIAL_ADMIN_PASSWORD` va rimossa dal file `.env` dopo il primo avvio.

Il certificato TLS va in `/opt/scarlet-server/tls/` con permessi 0600. SCARLET non serve mai HTTP in chiaro in produzione: nginx reindirizza la 80 sulla 443 e il cookie di sessione è marcato **Secure**. I dettagli completi, comprese le installazioni senza rete e la procedura di aggiornamento, sono in [docs/installation.md](#).

7. Postazioni e macchine di build

PC degli operatori

Nulla da installare. Serve un browser aggiornato (Chrome, Edge o Firefox) e le credenziali personali. Chi automatizza da script usa un token API personale e un qualsiasi client HTTP: il token eredita i permessi dell'utente ed è tracciato nell'audit con il suo nome.

PC o pipeline del team applicativo

Serve solo a chi confeziona i pacchetti, non a chi li distribuisce. Il pacchetto è un `tar.gz` con un manifest: può essere prodotto dallo script incluso nel repository oppure da qualunque strumento che rispetti il contratto.

Da installare	Perché
Docker o Podman	costruire l'immagine dell'applicazione
<code>docker save/podman save</code>	solo se l'immagine viaggia dentro il pacchetto, per gli host senza accesso al registry
Python 3.12+ e il repository SCARLET	per <code>scripts/build-scarlet-package.py</code> , che valida il pacchetto con le stesse regole dell'upload

Far girare il packager nella pipeline conviene: applica gli stessi controlli che SCARLET eseguirà al caricamento (schema del manifest, percorsi, link, dimensioni), quindi gli errori emergono nella build del team e non davanti all'operatore che sta rilasciando.

PC di sviluppo di SCARLET

Riguarda solo chi modifica SCARLET stesso: Docker Desktop o Podman Desktop con compose, Python 3.12+ e git. Il comando `docker compose --profile dev up -d --build` avvia l'intero ambiente, inclusi due server Oracle Linux simulati con Podman per le prove di DEV e PROD.

8. Matrice dei flussi di rete

La tabella elenca tutti i flussi, compresi quelli opzionali. Le righe obbligatorie sono due: gli operatori verso SCARLET e SCARLET verso gli host. Tutto il resto dipende da come è configurato l'ambiente.

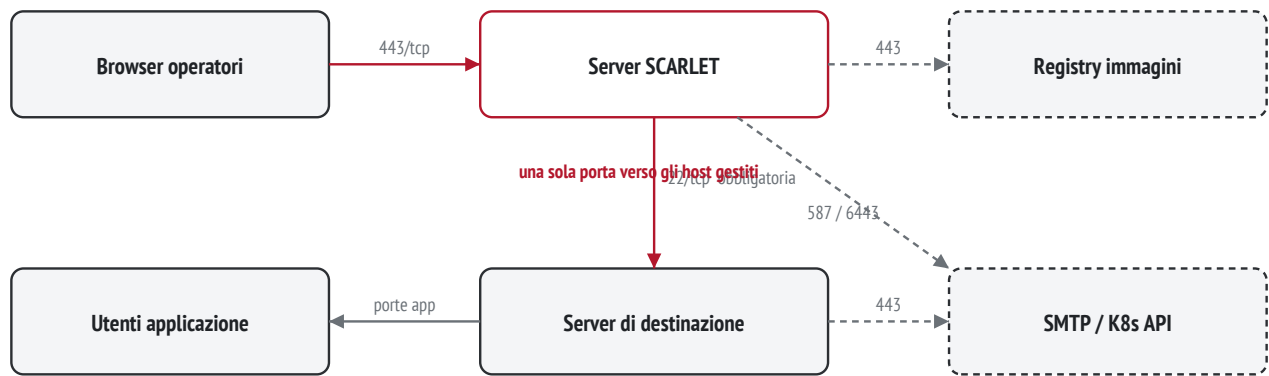


Figura 3 – I flussi principali. Le linee tratteggiate sono opzionali.

Da	Verso	Porta	Scopo	Obbl.
Browser degli operatori	Server SCARLET	443/tcp 80/tcp solo redirect	console web e API	sì
Server SCARLET (container web e worker)	ogni server di destinazione	22/tcp o la porta SSH configurata	SSH e SFTP: deploy, operazioni di ciclo di vita, discover, riconciliazione, log	sì
Server SCARLET	API server Kubernetes	6443/tcp	solo host Kubernetes in modalità API; con kubectl via SSH basta la 22	no
Server di destinazione	registry delle immagini	443/tcp	podman pull quando l'immagine non viaggia dentro il pacchetto	no
Server di destinazione	dipendenze applicative (database, code, API)	specifiche dell'app	le applicazioni distribuite, non SCARLET	dipende
Utenti e sistemi client	server di destinazione	porte delle applicazioni	uso normale delle applicazioni distribuite	sì
Server SCARLET	relay SMTP	587/tcp oppure 25 o 465	notifiche e-mail, solo se abilitate	no
Sistema di monitoraggio	Server SCARLET	443/tcp	/api/metrics, /api/health, /api/ready	no
Server SCARLET	repository del sistema operativo e registry	443/tcp	installazione e aggiornamenti del control plane	in fase di installazione
Server SCARLET	cdn.jsdelivr.net	443/tcp	asset grafici in modalità CDN; non serve con SCARLET_ASSET_MODE=local	no

Quello che non serve aprire

Due conseguenze pratiche che fanno risparmiare regole inutili. Primo: gli health check delle applicazioni partono **dal server di destinazione stesso** verso **127.0.0.1**, quindi non serve alcun flusso da SCARLET verso le porte applicative. Secondo: i server di destinazione non devono mai raggiungere SCARLET, in nessuna circostanza.

I flussi interni al server SCARLET (nginx verso gunicorn sulla 8000, gunicorn e worker verso PostgreSQL sulla 5432 e Redis sulla 6379) restano dentro la rete dei container e non richiedono alcuna regola sul firewall.

9. Errori tipici e come risolverli

Sono i casi che si incontrano davvero durante la preparazione di un host, con il messaggio esatto da cercare nei log e la correzione.

Sintomo o messaggio	Causa	Rimedio
<code>newuidmap: Operation not permitted</code>	l'utente non ha intervalli subuid/subgid	<code>usermod --add-subuids 100000-165535 --add-subgids 100000-165535 scarlet</code> , poi <code>podman system migrate</code>
il container sparisce pochi secondi dopo un deployment riuscito	manca il linger: systemd chiude la sessione utente	<code>loginctl enable-linger scarlet</code>
<code>stat /home/scarlet/.config: no such file or directory</code>	directory XDG mancanti alla prima esecuzione rootless	creare <code>.config</code> , <code>.local/share</code> e assicurarsi che <code>XDG_RUNTIME_DIR</code> sia valorizzato nella sessione
<code>controller 'pids' is not available</code>	cgroup non disponibili, tipico degli host annidati o dei container di prova	impostare <code>cgroups = "disabled"</code> in <code>~/.config/containers/containers.conf</code> ; su un host fisico o virtuale normale non serve
Discover riporta runtime NONE	il runtime non è installato o l'utente non riesce a eseguirlo	verificare con <code>sudo -iu scarlet podman info</code> ; con Docker controllare l'appartenenza al gruppo
Test connessione fallisce con errore di autenticazione	chiave non installata, permessi errati o restrizione <code>from=</code> non corrispondente	<code>chmod 700 ~/.ssh; chmod 600 authorized_keys; restorecon -Rv ~scarlet/.ssh</code> ; verificare l'IP di origine
stato della chiave host MISMATCH	la chiave del server è cambiata: reinstallazione oppure attacco	accertare la causa fuori banda; solo se legittima, riapprovare il nuovo fingerprint
pre-flight rosso su disk o memory	risorse libere inferiori a quelle richieste dal manifest	liberare spazio (<code>podman image prune -a</code>) o rivedere i requisiti dichiarati
pre-flight rosso su ports	la porta dell'applicazione è già occupata sull'host	liberare la porta o cambiarla nel manifest; in rootless ricordare il limite delle porte sotto la 1024
permission denied sui volumi del container	proprietà o etichetta SELinux errate	correggere il proprietario e lasciare che SCARLET monti con <code>:Z</code> ; <code>restorecon -Rv /opt/scarlet</code>
no space left on device durante il rilascio	storage delle immagini pieno	<code>podman system df</code> per capire, poi <code>podman image prune -a --filter until=168h</code>

10. Checklist di consegna

Da allegare alla consegna di ogni server di destinazione. Se tutte le voci sono spuntate, l'host può essere registrato in SCARLET e usato per un rilascio.

Fatto	Voce	Come si verifica
☐	Sistema operativo supportato e aggiornato	<code>cat /etc/oracle-release</code>
☐	Runtime installato e funzionante per l'utente dedicato	<code>sudo -iu scarlet podman run --rm busybox echo ok</code>
☐	Utilità presenti: tar, gzip, sha256sum, curl, nc	<code>command -v tar gzip sha256sum curl nc</code>
☐	Utente dedicato creato, senza sudo	<code>id scarlet</code>
☐	subuid e subgid assegnati	<code>grep '^scarlet:' /etc/subuid /etc/subgid</code>
☐	Linger abilitato	<code>loginctl show-user scarlet --property=Linger</code>
☐	Directory base scrivibile	<code>sudo -iu scarlet test -w /opt/scarlet && echo ok</code>
☐	Chiave pubblica installata con restrizione from=	<code>cat ~scarlet/.ssh/authorized_keys</code>
☐	SELinux enforcing	<code>getenforce</code>
☐	Firewall: SSH consentita solo dal server SCARLET	<code>firewall-cmd --list-all</code>
☐	Porte applicative aperte verso gli utenti	<code>firewall-cmd --list-ports</code>
☐	Fingerprint annotato e comunicato a chi lo approverà	<code>ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub</code>
☐	Orologio sincronizzato	<code>timedatectl</code>
☐	Test connessione e Discover riusciti dalla console web	pagina dell'host in SCARLET

Host: _____ Ambiente: _____ Data: _____

Preparato da: _____ Verificato da: _____

Appendice A. Elenco completo dei binari richiesti

Questa è la allow-list compilata dentro SCARLET: nessun altro eseguibile può essere invocato su un host, nemmeno da un amministratore. Serve come elenco esatto di ciò che deve esistere sul server e come garanzia sul perimetro delle operazioni.

Gruppo	Binari	Fornito da	Uso
Ispezione, sola lettura	<code>cat</code> <code>uname</code> <code>nproc</code> <code>free</code> <code>df</code> <code>id</code> <code>hostname</code> <code>readlink</code> <code>ls</code> <code>stat</code> <code>test</code> <code>sha256sum</code> <code>command</code> <code>which</code> <code>systemctl</code> <code>getenforce</code> <code>true</code>	coreutils, systemd, policycoreutils	Discover, pre-flight, verifica dei checksum
Filesystem, solo sotto la directory base	<code>mkdir</code> <code>mv</code> <code>rm</code> <code>ln</code> <code>tar</code> <code>cp</code> <code>chmod</code>	coreutils, tar	preparazione del layout, estrazione, attivazione della release
Runtime dei container	<code>podman</code> <code>docker</code> <code>kubectrl</code> <code>helm</code>	il runtime scelto	install, start, stop, restart, logs, stato, rollback
Health check	<code>curl</code> <code>nc</code>	curl, nmap-ncat	sonde HTTP e TCP eseguite dall'host su 127.0.0.1
Uso ristretto	<code>sh</code> <code>timeout</code> <code>find</code>	coreutils, findutils	utilizzabili solo attraverso costruttori interni con forma fissa degli argomenti

Su un'installazione minima di Oracle Linux mancano normalmente solo `nc` (pacchetto `nmap-ncat`) e il runtime. Tutto il resto è già presente. `getenforce` viene usato se disponibile e la sua assenza non è un errore.

Appendice B. Documenti di riferimento

Documento	Contenuto
docs/installation.md	installazione completa del control plane, TLS, systemd, backup, aggiornamenti, installazioni senza rete
docs/ssh.md	requisiti dell'utente remoto, autenticazione, verifica della chiave host, parametri di connessione
docs/runtime-podman.md	Podman rootless e rootful, riavvio dopo il reboot, storage e pulizia
docs/runtime-docker.md	specificità di Docker Engine
docs/runtime-kubernetes.md	modalità kubectl e API, permessi del service account
docs/RUNBOOK.md	31 procedure operative numerate con precondizioni, errori possibili e ripristino
docs/APPLICATION_RELEASE_CONTRACT.md	contratto del pacchetto e schema del manifest per i team applicativi
docs/security.md	modello delle minacce e controlli
Guida utente in linea	voce Guida utente nel menu: procedura completa dall'host vuoto all'applicazione online

Questo documento è generato dal repository di SCARLET con `python scripts/build_pdf_docs.py`, così resta allineato al codice: l'elenco dei binari, i percorsi e i parametri provengono dalle stesse fonti usate dall'applicazione.